ALGOPILOT
(algorithm visualizer and execution platform)

Submitted in partial fulfilment of the requirements of the degree of

BACHELOR OF ENGINEERING

by

Pratiksha Pathak: 22106127

Disha Waghmare: 22106033

Shikshita Yadav: 22106092

Guide:

Prof. Dr. Jaya Gupta

Department of Computer Science & Engineering

(Artificial Intelligence & Machine Learning)

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

(2025-2026)

Parshvanath Charitable Trust's
A. P. SHAH INSTITUTE OF TECHNOLOGY
(Approved by AICTE New Delhi & Govt. of Maharashtra, Affiliated to University of Mumbai)
(Religious Jain Minority)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

CERTIFICATE

This is to certify that the project entitled **"Algopilot(algorithm visualizer and execution platform)"** is a bonafide work of **Pratiksha Pathak(22106127), Disha Waghmare(22106033), Shikshita Yadav(22106092)** submitted to the University of Mumbai in partial fulfilment of the requirement for the award of the degree of **Bachelor of Engineering** in **Computer Science & Engineering (Artificial Intelligence & Machine Learning).**

Dr. Jaya Gupta Guide	Prof. Sayali P. Badhan Project Coordinator
Dr. Jaya Gupta Head of Department	Dr. Uttam D. Kolekar Principal

Parshvanath Charitable Trust's
A. P. SHAH INSTITUTE OF TECHNOLOGY
(Approved by AICTE New Delhi & Govt. of Maharashtra, Affiliated to University of Mumbai)
(Religious Jain Minority)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

Project Report Approval for B.E.

This project report entitled ***"Algopilot(algorithm visualizer and execution platform)"*** by **"Pratiksha Pathak, Disha Waghmare, Shikshita Yadav"** is approved for the degree of ***Bachelor of Engineering*** in ***Computer Science & Engineering (Artificial Intelligence & Machine Learning), 2025-26***.

Examiner Name	Signature
1._____	_____
2._____	_____

Date:
Place:

Declaration

We declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Pratiksha Pathak(22106127)

Disha Waghmare(22106033)

Shikshita Yadav(22106092)

Date:

Abstract

This project presents AlgoPilot, an Android-based mobile application developed to enhance the learning of Data Structures and Algorithms (DSA) through interactive visualization. Traditional teaching methods often make it difficult for students to follow the logical flow of algorithms, leading to gaps in understanding. To address this, AlgoPilot integrates custom-built animations synchronized with pseudocode to illustrate algorithm execution step by step, without relying on prebuilt visualization libraries. The application allows learners to control execution speed, navigate forward or backward, and observe algorithm behavior in real time, thereby bridging the gap between theoretical concepts and practical understanding. With its offline accessibility, modular design, and student-friendly interface, AlgoPilot demonstrates the potential of mobile-based visualization tools to improve clarity, engagement, and accessibility in computer science education. In alignment with the United Nations Sustainable Development Goals (SDGs), AlgoPilot supports SDG 4 (Quality Education) by promoting interactive, self-paced algorithm learning for all, contributes to SDG 9 (Industry, Innovation, and Infrastructure) by leveraging mobile technology to modernize computer science education, and advances SDG 10 (Reduced Inequalities) by providing offline access that benefits students in remote or low-resource areas.

Keywords: Algorithm Visualization, Data Structures and Algorithms (DSA), Mobile Learning, Interactive Education, Pseudocode Synchronization, Offline Accessibility, Modular Design, Educational Technology, Self-paced Learning

CONTENTS

1. Introduction…..... 1
2. Literature Survey…….3
3. Limitation of Existing System……....9
4. Problem Statement, Objectives and Scope….....10
5. Proposed System...11
6. Experimental Setup……...13
7. Project Plan……20
8. Implementation….....22

References…….....26

LIST OF FIGURES

5.1. Proposed System.....13
5.2. Data Flow Diagram 14
5.3. Use Case Daigram.....15
7.1. Gantt Chart.....20
8.1. Homepage 21
8.2. Sorting Algorithms.....21
8.3. Sorting Eaxmple 22
8.4. Tree Algorithm.....22
8.5. Tree Example.....23
8.6. Graph Algorithms.....23
8.7. Graph Example 24

LIST OF TABLES

2.1. Literature Survey.....7
2.1. Performance Analysis.....27

Chapter 1
Introduction

The AlgoPilot app enhances the teaching and learning of Data Structures and Algorithms (DSA) by moving beyond traditional methods such as textbooks, printed pseudocode, and static lectures. Conventional approaches often make it difficult for students to follow the logical flow of algorithms and understand abstract concepts, which can lower motivation and limit problem-solving skills. By introducing interactive visualizations and real-time animations, AlgoPilot bridges the gap between theory and practice, helping learners develop stronger analytical abilities and retain key concepts for the long term.

At its core, AlgoPilot is a mobile-based algorithm visualization platform designed for simplicity, accessibility, and effectiveness. The app includes features such as adjustable animation speed, forward and backward navigation, synchronized pseudocode highlighting, and a clean, intuitive interface. Covering a variety of topics like sorting, tree traversals, and graph algorithms, AlgoPilot allows students to explore complex ideas at their own pace, transforming algorithmic logic into clear, step-by-step visual experiences that make learning both interactive and enjoyable.

Beyond visualization, AlgoPilot addresses critical challenges of accessibility and scalability by supporting full offline functionality and a modular design. This ensures uninterrupted learning for students in remote or resource-constrained areas and allows new algorithms to be added as academic requirements evolve. By combining interactivity, offline access, and a user-friendly design, AlgoPilot empowers learners to build confidence, strengthen problem-solving skills, and

stay curious, representing a significant step forward in reimagining how algorithms are taught in the digital age.

Sustainability Development Goal Alignment:

- **SDG 4 – Quality Education:**
  AlgoPilot promotes inclusive and high-quality education by enabling interactive, self-paced learning of Data Structures and Algorithms. Through engaging visualizations and real-time animations, it helps students understand complex concepts more effectively, improving both comprehension and retention.
- **SDG 9 – Innovation & Infrastructure:**
  By leveraging mobile technology and custom-built visualizations, AlgoPilot modernizes the way computer science is taught and learned. It encourages innovation in educational infrastructure by providing a scalable platform that integrates technology with advanced teaching methods.
- **SDG 10 – Reduced Inequalities:**
  AlgoPilot ensures equitable access to education by offering full offline functionality, allowing students in remote or low-resource areas to learn without dependence on internet connectivity or expensive equipment. This reduces barriers and supports equal learning opportunities for all.

Chapter 2

Literature Survey

1. **Kumari, A., Mittal, M., Jha, V., Sahu, A., Kumar, M., Sangwan, N., & Bohra, N. (2022). Algorithm visualisation—modern web-based visualization of sorting and searching algorithms.** ***Advances and Applications in Mathematical Sciences,*** *21*(9), 4441–4452**.**

   Kumari, A., Mittal, M., Jha, V., Sahu, A., Kumar, M., Sangwan, N., & Bohra, N. (2022) publish Algorithm Visualization—Modern Web-Based Visualization of Sorting and Searching Algorithms in Advances and Applications in Mathematical Sciences.

   The paper presents a web-based platform to visualize sorting and searching algorithms, improving conceptual clarity, but it is dependent on internet access and not designed for mobile devices

2. **Lanjewar, R., Mehar, N., Doshi, A., & Mane, Y. (2022). AlgoVis—an enhanced way to visualise algorithms. In** ***ITM Web of Conferences: International Conference on Advances in Computing and Communication (ICACC)*****. EDP Sciences.**

   Lanjewar, R., Mehar, N., Doshi, A., & Mane, Y. (2022) describe AlgoVis—An Enhanced Way to Visualize Algorithms in ITM Web of Conferences: ICACC.

   The system enhances algorithm visualization through a better user interface and interactive controls, but it remains web-based and lacks offline or mobile-focused features.

3. **Sharma, P., & Verma, K. (2024). The efficacy of animation and visualization in teaching data structures: A case study. *Journal of Computer Science Education Research, 15*(1), 55–68.**

   Sharma and Verma (2024) in Journal of Computer Science Education Research, 15(1), 55–68 present The Efficacy of Animation and Visualization in Teaching Data Structures: A Case Study.

   The study shows that animated algorithm visualization significantly improves student understanding and engagement. However, it focuses on specific algorithms and offers no modular expansion or offline mobile platform.

4. **Zhang, Y. (2024). Application of algorithm visualization techniques in teaching computer data structure course. *International Journal of Education and Development using ICT, 20*(2), 113–125.**

   The study demonstrates that integrating real-time visualizations into classroom instruction improves students' understanding and engagement with complex data structure concepts. However, the approach is primarily desktop-based and requires constant internet connectivity, lacking a mobile-friendly or offline learning environment.

5. **Bhandari, S. R., Abhijeet, C., Pratik, B., & Adinath, T. (2025). Algorithm visualization on data structures along with AI chatbot. *International Journal of Innovative Research in Computer Science & Technology (IJIRCST), 13*(1), 21–27.**

   Bhandari, S. R., Abhijeet, C., Pratik, B., and Adinath, T. (2025) present a web-based tool that combines algorithm visualization with an AI chatbot to help students understand data structure algorithms through step-by-step animations and real-time guidance. While effective for engagement, it is limited to online use,

lacks offline access, and is not designed as a mobile-first or easily expandable platform.

6. **Liu, H. (2024). Research and application of animation visualization based on HTML5 algorithm. *Journal of Web Technologies, 12*(3), 45–53.**
   This paper resents a study titled Research and Application of Animation Visualization Based on HTML5 Algorithm in the Journal of Web Technologies, 12(3), 45–53. The paper introduces an HTML5-based platform for real-time animation of algorithms to aid teaching and learning. It effectively demonstrates how web-based animations enhance understanding, but it remains a browser-only solution and lacks mobile-first or offline support.

7. **Guixiang Lou, Wang Sufang, and Hongchun Shen. (2024). Research on graphic interactive visual online teaching of intelligent algorithm course. In *Proceedings of BigIoT-EDU 2024*. IEEE.**
   Guixiang Lou, et.al., (2024) in Proceedings of BigIoT-EDU 2024 (IEEE) discusses Research on Graphic Interactive Visual Online Teaching of Intelligent Algorithm Course. The work proposes an interactive online teaching system for intelligent algorithms using graphic visualizations. While it promotes engagement and interactivity, the system is entirely online, requiring internet access and lacking offline or mobile adaptability.

8. **Singh, R., & Patel, A. (2023). Demystifying complex algorithms through interactive visualizations. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education (SIGCSE TS 2023)* (pp. 401–407). ACM.**
   Singh and Patel (2023) in Proceedings of the 54th ACM Technical Symposium

on Computer Science Education (SIGCSE TS 2023), pp. 401–407 explore Demystifying Complex Algorithms Through Interactive Visualizations. The paper introduces an interactive web framework to visualize complex algorithms like dynamic programming. It successfully simplifies advanced topics but is limited to a web-based environment without mobile or offline capabilities.

9. **Gupta, V., & Rao, D. (2023). Visualizing algorithms in the field of education: A comprehensive review. *International Journal for Research in Applied Science and Engineering Technology (IJRASET), 11*(7), 250–258.**
   Gupta and Rao (2023) present Visualizing Algorithms in the Field of Education: A Comprehensive Review in the International Journal for Research in Applied Science and Engineering Technology (IJRASET), 11(7), 250–258.
   This comprehensive review surveys various algorithm visualization tools and their impact on education. While informative, it is purely a review and lacks implementation or a dedicated mobile-friendly solution.

10. **Kocakoyun, Ş. *(2019)*. Developing of Android Mobile Application Using Java and Eclipse: An Application. *International Journal of Electronics Mechanical and Mechatronics Engineering*,** 7**(1), 1335–1354**
    Kocakoyun (2019) in International Journal of Electronics Mechanical and Mechatronics Engineering, 7(1), 1335–1354 presents Developing of Android Mobile Application Using Java and Eclipse: An Application. This paper discusses the process of building Android applications using Java and Eclipse, providing insights into mobile app development. While useful for app creation, it is not specifically focused on algorithm visualization or education.

Year	Title	Domain	Algorithm	Dataset	Gap
2022	Algorithm Visualization Modern Web-Based Visualization of Sorting &Searching Algorithms (Kumari et al.)	Web-based Visualization	Sorting and Searching	Not specified (web demos)	Limited to web; lacks mobile/offline access
2022	AlgoVis—An Enhanced Way to Visualize Algorithms (Lanjewar et al.)	Algorithm Visualization	Sorting, Searching	Demo datasets	Focuses on enhancement; no modular mobile design
2024	The Efficacy of Animation and Visualization in Teaching Data Structures: A Case Study	Computer Science Education	Sorting, Recursion, Graphs	Student performance data	Limited algorithm coverage, no offline support
2024	Application of Algorithm Visualization Techniques in Teaching Computer Data Structure Course (Zhang, 2024)	CS Education	Data Structures and Algorithms (various)	Classroom experiment datasets	Desktop-based teaching; lacks mobile/offline support

2025	Algorithm Visualization on Data Structures Along with AI Chatbot (Bhandari et al.)	Algorithm Visualization + AI	Sorting, Searching	Interactive chatbot – based examples	Web-based; no offline capability or modular expansion
2024	Research and Application of Animation Visualization Based on HTML5 Algorithm (Liu, 2024)	Web-based Visualization	General Algorithm Animation	Web-based environment	Uses web tech only; no mobile or modular extension
2024	Research on Graphic Interactive Visual Online Teaching of Intelligent Algorithm Course	Online Interactive Education	Intelligent Algorithms and DS	Online teaching platform	Requires online connectivity; not mobile/offline-centric
2023	Demystifying Complex Algorithms Through Interactive Visualizations (SIGCSE TS 2023)	Educational Visualization	Advanced Algorithms (e.g., Weighted Interval Scheduling, DP)	Web-based framework	Web-only; no mobile/offline features
2023	Visualizing Algorithms in the Field of Education: A Comprehensive Review (IJRASET)	Review paper	Sorting, Trees, Graphs	NA	Survey only; lacks practical implementation

2019	Developing of Android Mobile Application Using Java and Eclipse: An Application (Şenay Kocakoyun et al.)	Mobile Development App	Android activity lifecycle & event-driven programming	UI components, XML layouts, and sample inputs	Lack of practical, structured, academic guidance for beginners.

Table 2.1 Literature Survey

Chapter 3

Limitations of Existing Systems

1. **Web or Desktop Dependency**

- Most existing visualization tools (e.g., *Algoviz, AlgoVis, HTML5-based animations*) are web/desktop dependent.
- This limits accessibility for students without constant internet or high-performance PCs.

2. **Lack of Offline Access**

- Very few systems support offline functionality (*only some early mobile attempts*).
- Students in remote or low-resource areas face challenges using these tools.

3. **Low Interactivity**

- Several visualizers only show animations passively (play-only).
- Features like step-by-step navigation, adjustable speed, or pseudocode sync are missing in most.

4. **User Interface Complexity**

- Some systems are designed by researchers for academic demonstrations, leading to complex UIs that aren't student-friendly.
- Lack of focus on simplicity and intuitiveness for beginners.

5. **Scalability Issues**

- Review papers highlight that tools aren't built with scalability in mind.
- They are prototypes, not robust educational platform

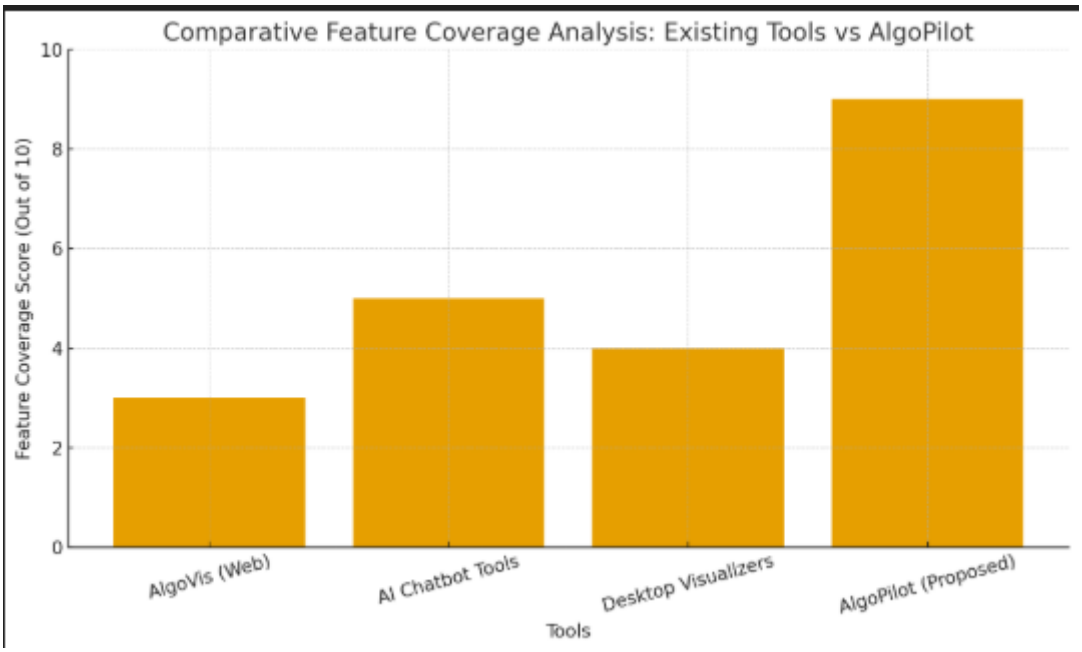

Figure 3.1. Existing Systems Vs AlgoPilot

Chapter 4
Problem Statement, Objectives and Scope

Problem Statement:

Traditional methods like textbooks and static pseudocode for learning Data Structures and Algorithms (DSA) is often challenging and fail to convey the step-by-step logic and real-time execution of algorithms. Existing tools are mostly web or desktop-based, limited in scope, and lack offline accessibility or modularity, making them less effective for students. To overcome these issues, AlgoPilot is designed as a mobile application that provides interactive, step-by-step visualizations, synchronized pseudocode, and real-time animations, enabling learners to intuitively understand algorithm flow in an accessible and engaging way.

Objectives:

- To provide an interactive, mobile-friendly platform for students to visualize and explore algorithms step-by-step, making abstract concepts easier to understand and encouraging effective, self-paced learning anytime and anywhere. The mobile-first approach ensures that learners can study independently without relying on formal classroom setups or traditional teaching methods.
- To bridge the gap between theoretical knowledge and practical understanding by converting complex algorithmic logic into real-time visual simulations. This helps students grasp how each line of code translates into action, enabling them to develop a deeper conceptual foundation and apply their learning more confidently in problem-solving scenarios.

- To ensure a scalable and accessible design that supports modular expansion of new algorithms and features over time. The application is built to function without the need for desktop tools or continuous internet access, thereby making algorithm learning available to a broader audience, including those in resource-constrained or remote environments.

Scope:

- Educational Reach – AlgoPilot is designed primarily for students learning Data Structures and Algorithms (DSA), but it can also support teachers, trainers, and self-learners who want a practical, visual aid to improve teaching and understanding.
- Algorithm Coverage – The application will initially focus on fundamental sorting, trees and graphs algorithms but will be scalable to include advanced concepts like searching and dynamic programming in future versions.
- Interactive Features – The tool provides step-by-step execution, adjustable speed controls, pseudocode highlighting, and replay options, making it more interactive than static tools or lectures.
- Offline Functionality – Unlike most web tools, AlgoPilot will support offline use, ensuring learning is not restricted by network availability, which is vital for students in low-connectivity regions.
- User Experience (UX) – The project scope includes creating a clean, intuitive, and user-friendly interface that reduces cognitive load and makes algorithm learning engaging and enjoyable.
- Scalability & Future Enhancements – AlgoPilot's modular design allows easy addition of new algorithms, AI-assisted explanations, quizzes, and chatbot integration in later stages to broaden its educational value.

- Contribution to Education – The project not only aids in academic learning but also prepares students for competitive coding, technical interviews, and real-world problem solving, bridging the gap between classroom learning and industry needs.

Chapter 5
Proposed System

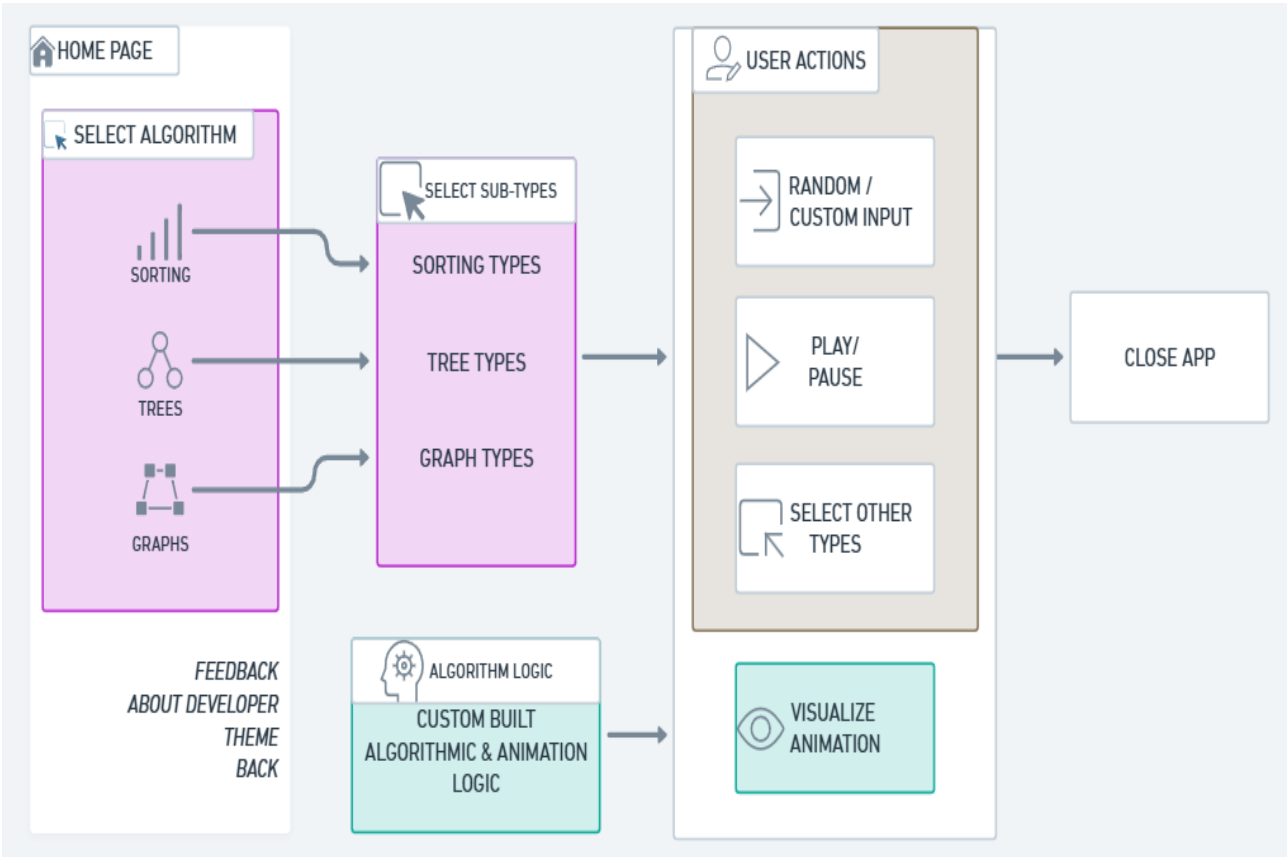

Figure 5.1. Proposed System

1. Data Flow Diagram (DFD)

The Data Flow Diagram represents the logical flow of data within the Algorithm Visualization System. It outlines how user inputs are processed and transformed into meaningful visual outputs.

The process begins on the Home Page, where the user selects a specific algorithm category. This selection is passed to the Algorithm Logic Module, which interprets the input and executes the corresponding algorithm using pre-defined logic. The

computed results are then transmitted to the Visualization Module, responsible for rendering dynamic, step-by-step animations of the algorithm in execution.

Throughout this process, the user can interact with the system through actions such as Play**,** Pause**,** Restart**,** or Change Input, thereby enhancing engagement and 0learning. Once the visualization is complete, the user can terminate or return to the main interface.

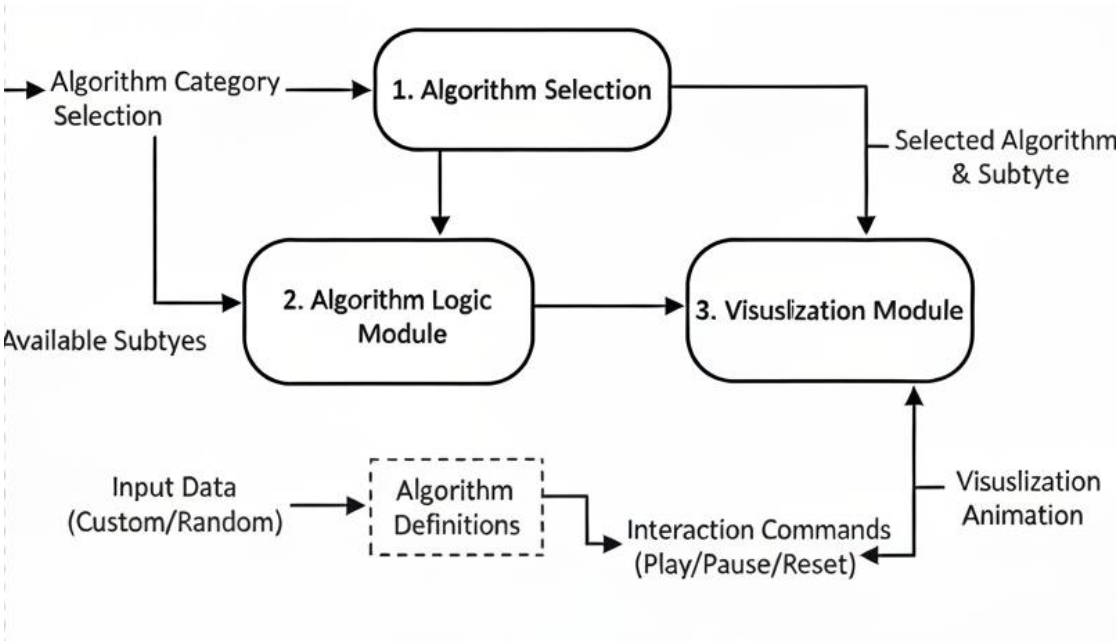

Figure 5.2. Data Flow Diagram

Flow Summary: *User Input → Algorithm Logic Module → Processing → Visualization Module → User Interaction → Output Display*

2. Use Case Diagram

The Use Case Diagram depicts the relationship between the User and the Algorithm Visualization System, highlighting the functional aspects and possible interactions.

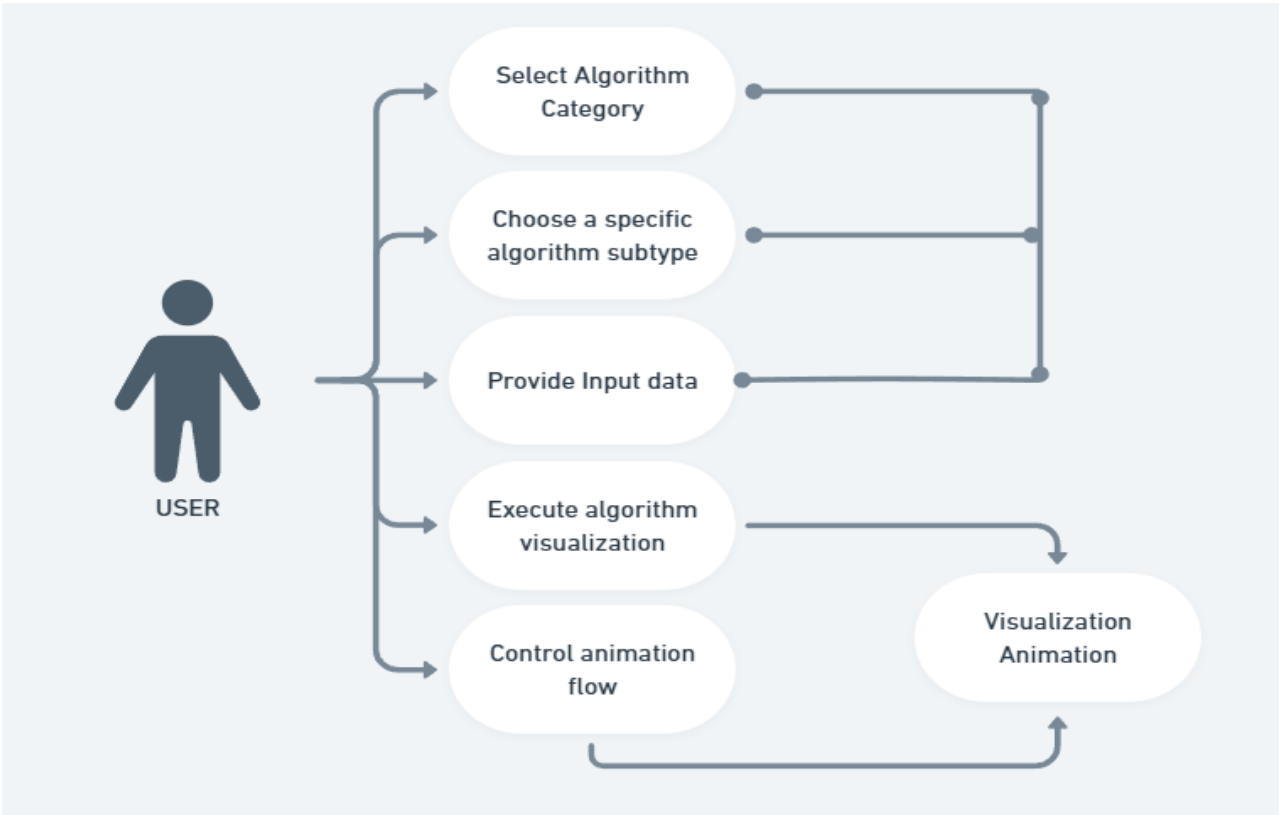

Figure.5.3. Use Case Diagram

Primary Actor: User

System: Algorithm Visualization Application

Key Use Cases:

1. Select algorithm category (e.g., Sorting, Tree, Graph)
2. Choose a specific algorithm subtype
3. Provide input data (custom or randomly generated)
4. Execute algorithm visualization
5. Control animation flow (Play, Pause, Stop, Reset)
6. View real-time graphical representation of the algorithm

This diagram emphasizes how users actively interact with the application to gain a clear understanding of algorithmic behaviour through visual learning.

3. Sequence Diagram

The Sequence Diagram presents a chronological representation of the interactions among system components during algorithm visualization.

Stepwise Flow:

1. The user selects the algorithm type from the Home Page interface.
2. The system retrieves and displays the available subtypes related to the chosen category.
3. The user provides input data or opts for system-generated sample data.
4. The Algorithm Logic Module executes the selected algorithm and processes the data.
5. The computed results are transferred to the Visualization Module, which generates real-time graphical animations.
6. The User Interaction Panel enables control over the visualization process, ensuring a seamless user experience.

This diagram outlines the sequential coordination between user actions and internal system operations, ensuring clarity in workflow and modular design.

4. Activity Diagram

The Activity Diagram provides a visual representation of the system's operational flow from initialization to completion.

Workflow:

Start → Select Algorithm → Choose Subtype → Provide Input → Execute Algorithm → Visualize Process → Interact (Play/Pause/Modify Input) → End

This diagram captures the complete user journey and system responses at each stage, ensuring that every process follows a structured and logical path.

5. Algorithm Workflow

The algorithmic process of the application follows a systematic and modular sequence:

1. **Start:** Initialize the system and load available algorithm categories.
2. **Select Category:** The user selects a major algorithm type (Sorting, Tree, or Graph).
3. **Choose Subtype:** The system presents subtypes (e.g., Quick Sort, Merge Sort).
4. **Input Data:** User enters custom data or allows automatic data generation.
5. **Execute Logic:** The Algorithm Logic Module applies the relevant algorithm to process input data.
6. **Generate Animation:** The Animation Engine visualizes each computational step in real-time.
7. **User Interaction:** The user may play, pause, or modify inputs during visualization.
8. **Display Result:** The system presents the final outcome and awaits user termination or navigation back to the home screen.

6. Key Components of the Proposed Framework

1. Home Page Interface: Serves as the entry point of the system, allowing users to navigate between different algorithm categories and subtypes through an intuitive layout.

2. Algorithm Logic Module: Implements algorithm-specific logic using optimized data structures. It ensures accurate computation and forms the foundation for animation generation.

3. Animation Engine: Transforms algorithmic computations into dynamic, step-by-step animations. It provides visual feedback that helps users understand algorithm behaviour intuitively.

4. User Interaction Panel: Offers control elements such as *Play, Pause* and *Change Input* for interactive visualization. Enhances learning by allowing users to explore algorithms at their own pace.

5. Feedback Section: Provides a channel for users to share experiences, suggest improvements, and provide developer feedback, promoting iterative refinement of the application.

Chapter 6

Experimental Setup

Platform

Android Studio:

Android Studio serves as the core development platform for building and testing the application. It provides powerful tools for code editing, debugging, UI design, and performance profiling, ensuring a streamlined development workflow.

- **Scope:** To enable end-to-end Android app development and testing within a unified environment.
- **Use Case:** Developers use Android Studio to implement algorithm visualizations, simulate animations, and test app behavior across multiple devices.
- **Focus:** Efficiency in development, debugging accuracy, and performance optimization for real-time animations.

Language

Java:

Java is the primary programming language used to implement the app's logic, algorithm animations, and user interaction layers. It ensures high compatibility with Android systems and supports structured, object-oriented programming for maintainable code.

- **Scope:** To handle algorithmic operations, visual logic simulations, and app navigation efficiently.
- **Use Case:** Defining algorithm workflows (like sorting or searching), rendering visual representations, and managing UI components.

- **Focus:** Stability, reusability, and seamless user interaction through efficient event handling.

Build Tool

Gradle:

Gradle automates the build process, testing, and packaging of the Android app. It manages dependencies and external libraries needed for animations, visualizations, and advanced UI elements.

- **Scope:** To ensure consistent, automated builds and dependency management.
- **Use Case:** Integrating third-party libraries for animations, UI enhancement, and version control during development.
- **Focus:** Build automation, dependency resolution, and smooth integration of external modules.

Software Requirements

- **Android SDK (latest stable version):** Provides the necessary APIs and tools for Android development.
- **JDK 8 or higher:** Ensures compatibility and optimal performance for Java-based Android applications.
- **Emulator or Physical Android Device:** Used to test responsiveness, performance, and user experience across devices.
  - **Scope:** Establish a stable foundation for app development and execution.
  - **Use Case:** Testing app compatibility, responsiveness, and visual performance.
  - **Focus:** Reliability, stability, and compatibility across environments.

Operating Environment

The app is designed to operate smoothly across various Android devices and screen resolutions, ensuring accessibility and usability.

- **Scope:** Deliver a responsive, user-friendly learning experience even without internet connectivity.
- **Use Case:** Enables students and learners to visualize algorithms in offline mode for uninterrupted study sessions.
- **Focus:** Offline functionality, responsiveness, and optimized real-time animation performance.

Development Environment

IDE: Android Studio

Version Control: Git / GitHub

Testing Tools: Android Emulator,

- **Scope:** To maintain a collaborative, version-controlled development process with continuous testing.
- **Use Case:** Team collaboration, tracking code changes, and evaluating performance metrics.
- **Focus:** Code maintainability, performance monitoring, and reliable testing workflows.

Chapter 7
Project Planning

Gantt Chart:

Figure 7.1 July (Gantt Chart)

Figure 7.2 August (Gantt Chart)

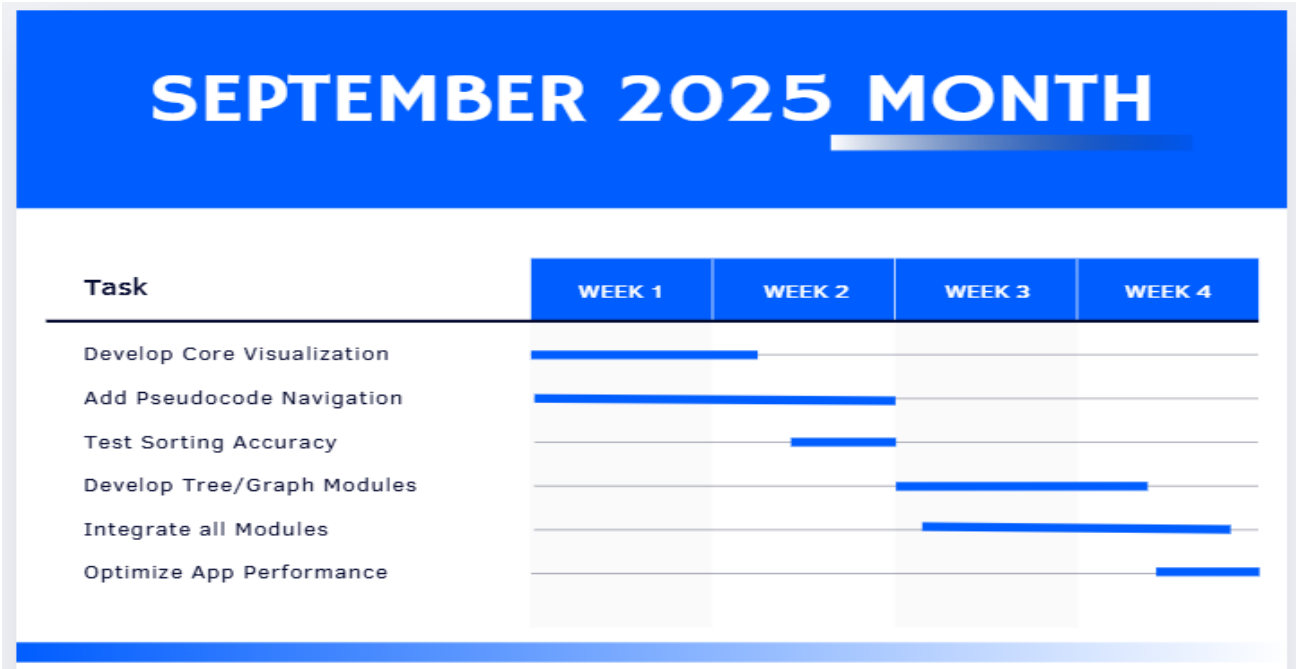

Figure 7.3 September (Gantt Chart)

Figure 7.4 October (Gantt Chart)

Chapter 8
Implementation

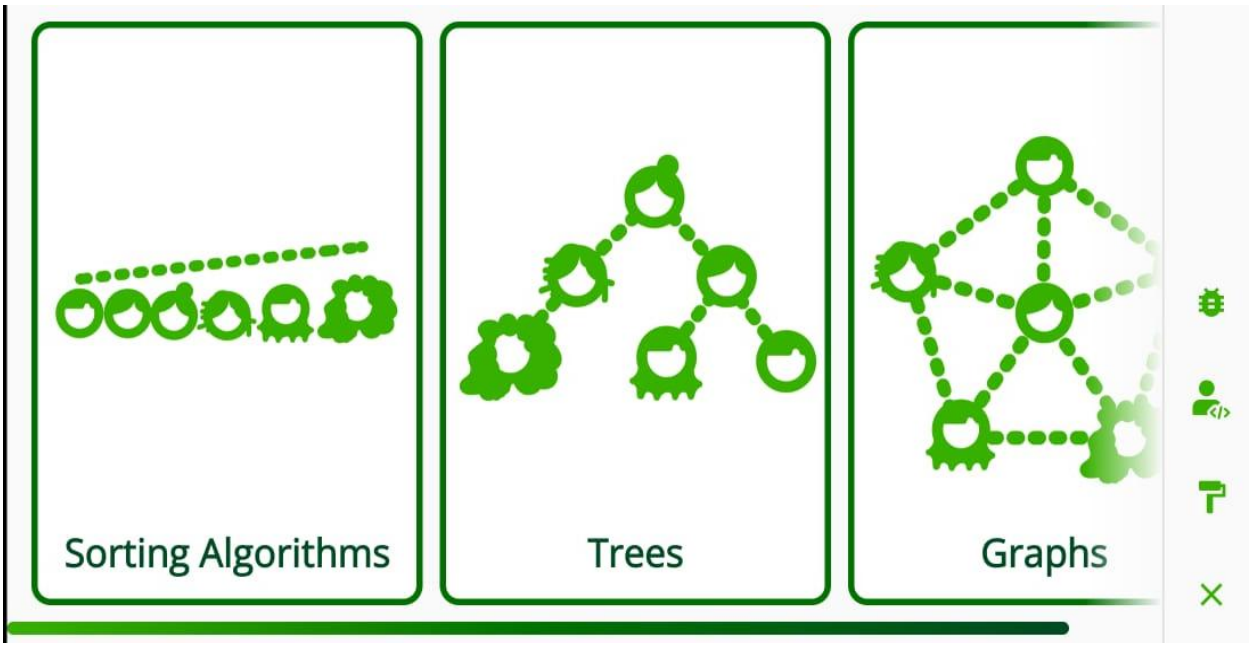

Figure 8.1 Home Page

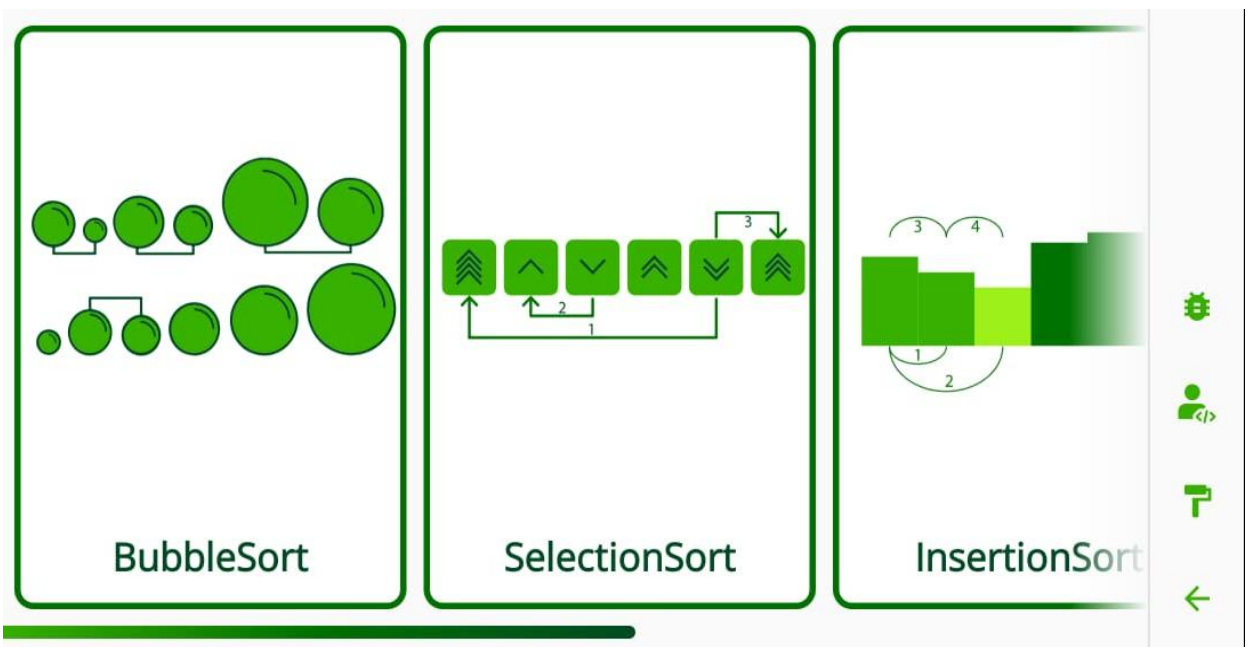

Figure 8.2 Sorting Algorithms

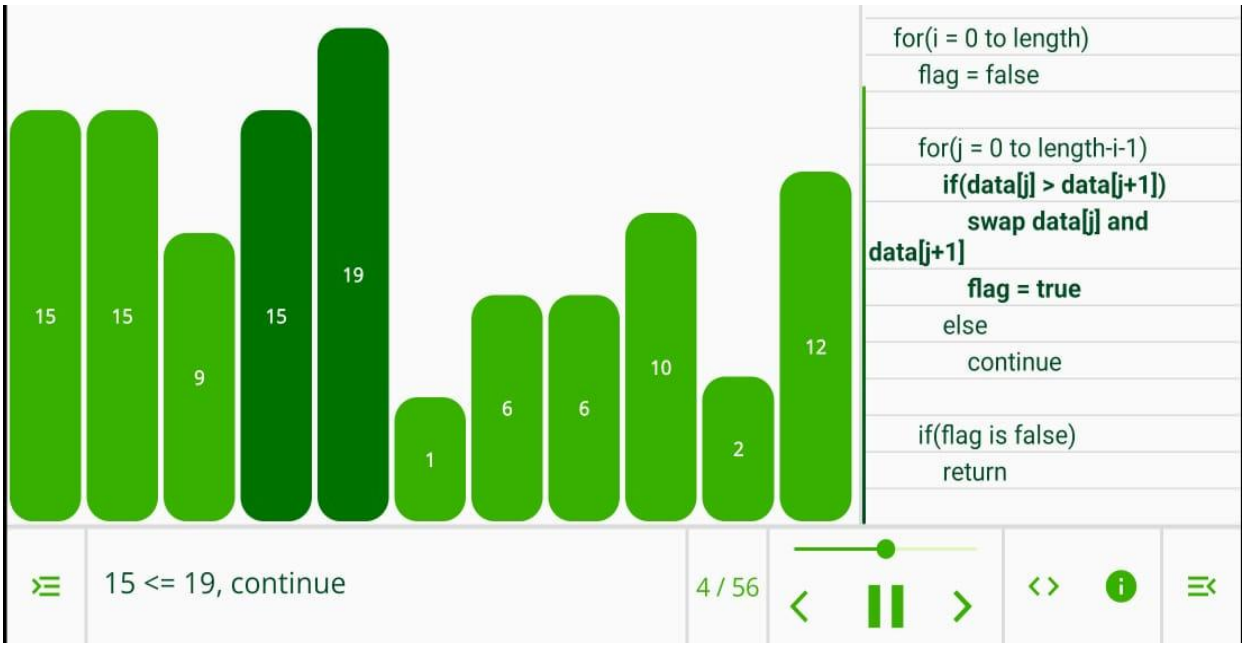

Figure 8.3 Sorting Example

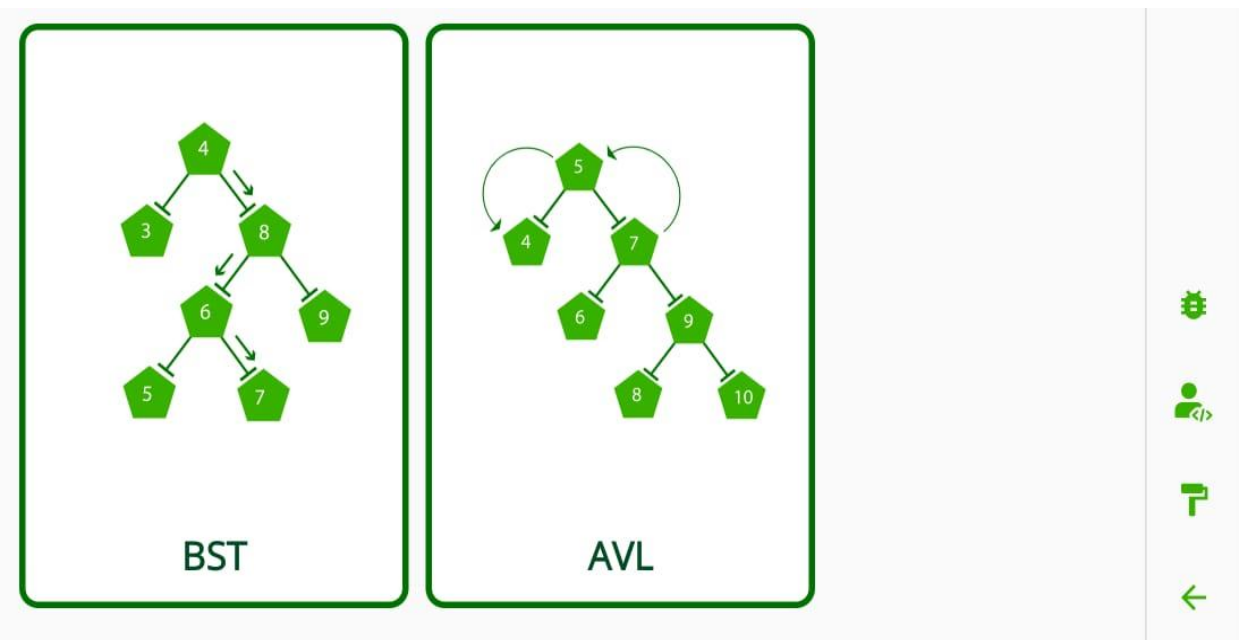

Figure 8.4 Tree Algorithms

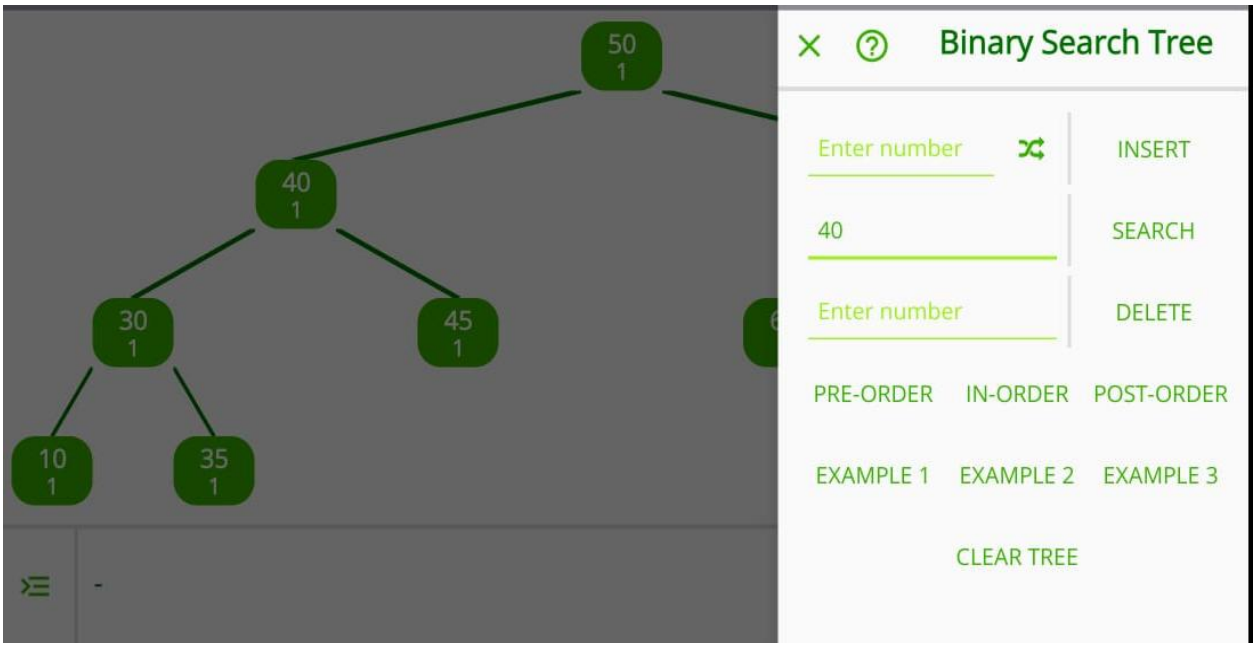

Figure 8.5 Tree Example

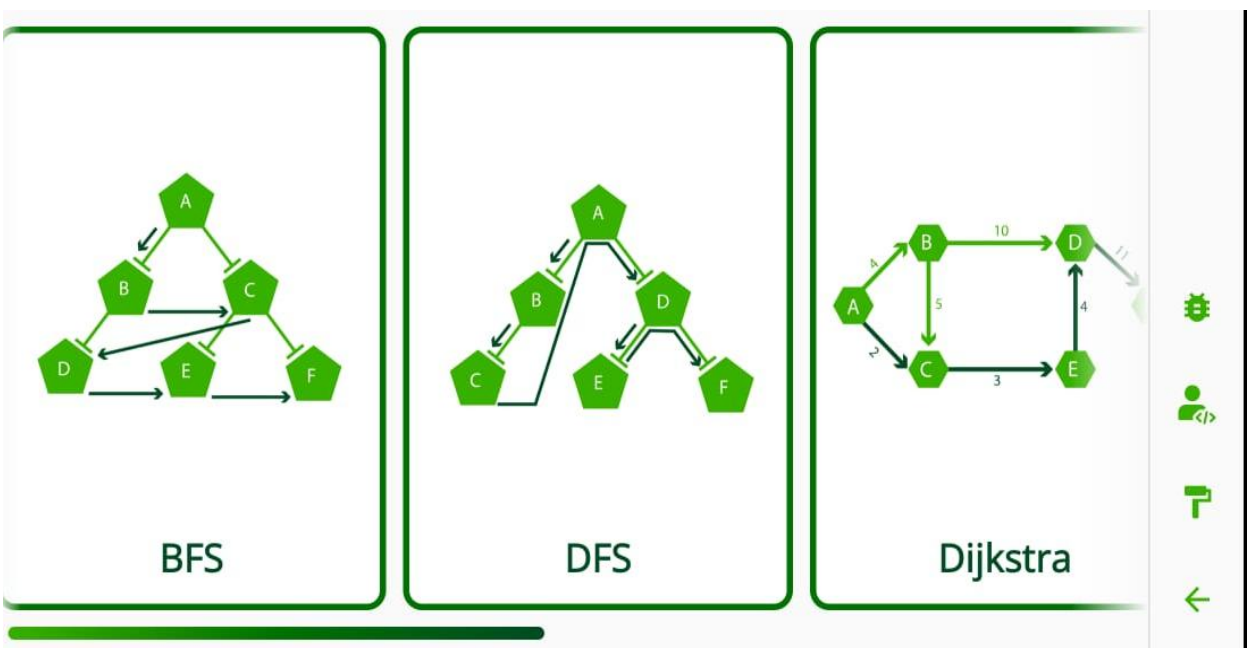

Figure 8.6 Graph Algorithms

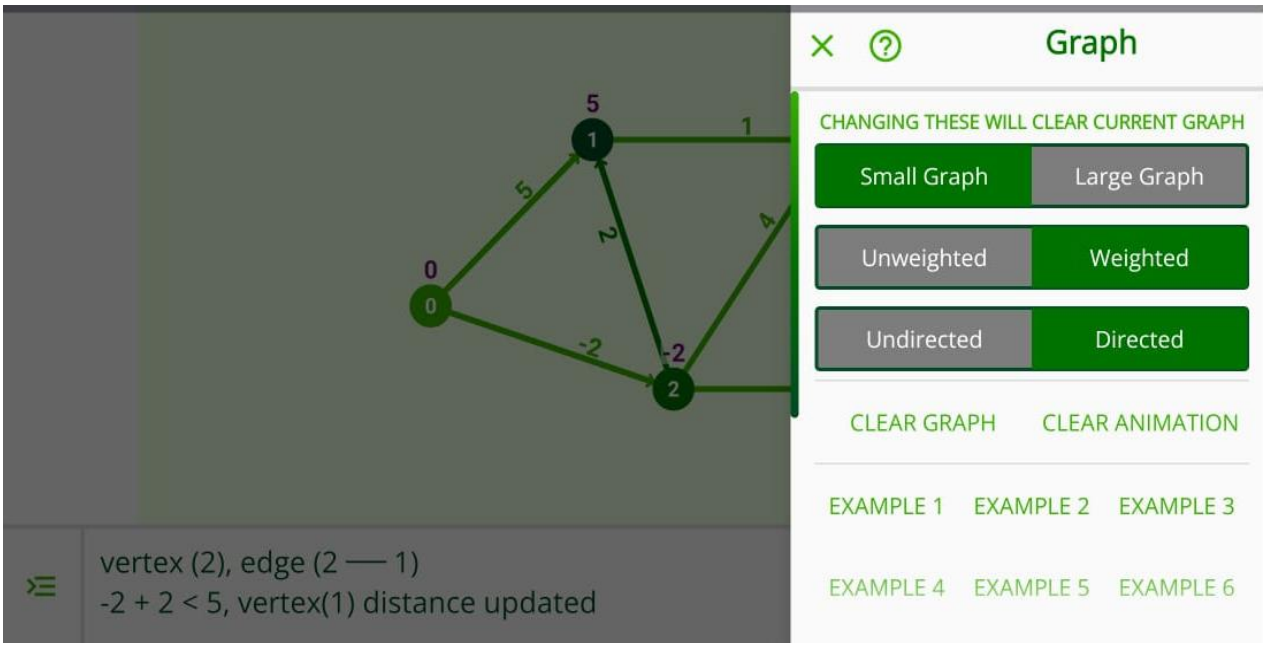

Figure 8.7 Graph Example

Algorithm	Time	Accuracy
Bubble Sort	Animation Speed + O(n^2)	100%
Selection Sort	Animation Speed + O(n^2)	100%
Insertion Sort	Animation Speed + O(n^2)	100%
Merge Sort	Animation Speed + O(n log n)	100%
Quick Sort	Animation Speed + O(n log n)	100%
Tree Traversal	Animation Speed + O(n)	100%
BFS (Graph Traversal)	Animation Speed + O(V+E)	100%
DFS (Graph Traversal)	Animation Speed + O(V+E)	100%

Table 8.1 Performance Analysis

REFERENCES

[1] Kumari, A., Mittal, M., Jha, V., Sahu, A., Kumar, M., Sangwan, N., & Bohra, N. (2022). Algorithm visualisation—modern web-based visualization of sorting and searching algorithms. *Advances and Applications in Mathematical Sciences, 21*(9), 4441–4452.

[2] Lanjewar, R., Mehar, N., Doshi, A., & Mane, Y. (2022). AlgoVis—an enhanced way to visualise algorithms. In *ITM Web of Conferences: International Conference on Advances in Computing and Communication (ICACC)*. EDP Sciences.

[3] Sharma, P., & Verma, K. (2024). The efficacy of animation and visualization in teaching data structures: A case study. *Journal of Computer Science Education Research, 15*(1), 55–68.

[4] Zhang, Y. (2024). Application of algorithm visualization techniques in teaching computer data structure course. *International Journal of Education and Development using ICT, 20*(2), 113–125.

[5] Bhandari, S. R., Abhijeet, C., Pratik, B., & Adinath, T. (2025). Algorithm visualization on data structures along with AI chatbot. *International Journal of Innovative Research in Computer Science & Technology (IJIRCST), 13*(1), 21–27.

[6] Liu, H. (2024). Research and application of animation visualization based on HTML5 algorithm. *Journal of Web Technologies, 12*(3), 45–53.

[7] Guixiang Lou, Wang Sufang, and Hongchun Shen (2024). Research on graphic interactive visual online teaching of intelligent algorithm course. In *Proceedings of BigIoT-EDU 2024*. IEEE.

[8] Singh, R., & Patel, A. (2023). Demystifying complex algorithms through interactive visualizations. In *Proceedings of the 54th ACM Technical*

Symposium on Computer Science Education (SIGCSE TS 2023) (pp. 401–407). ACM.

[9] Gupta, V., & Rao, D. (2023). Visualizing algorithms in the field of education: A comprehensive review. *International Journal for Research in Applied Science and Engineering Technology (IJRASET), 11*(7), 250–258.

[10] Kocakoyun, Ş. *(2019)*. Developing of Android Mobile Application Using Java and Eclipse: An Application. *International Journal of Electronics Mechanical and Mechatronics Engineering*, **7**(1), 1335–1354